

1 Introduction

Phishing is a cyberattack technique where attackers impersonate trusted entities through deceptive emails in order to steal sensitive information such as passwords, financial data, or personal credentials. With the rise of Large Language Models (LLMs), phishing emails have become significantly more sophisticated, grammatically correct, and contextually convincing, making traditional rule-based detection methods less effective. Modern phishing detection therefore increasingly relies on Artificial Intelligence (AI) and Machine Learning (ML) to identify subtle malicious patterns in email content and metadata.

1.1 Core Cybersecurity Concepts

- Phishing: social-engineering attacks that trick users into revealing credentials or clicking malicious links.
- Attack signals: urgency language, credential requests, fake verification flows, suspicious URLs, and reward scams.
- False Positive (FP): legitimate email predicted as phishing.
- False Negative (FN): phishing email predicted as legitimate.
- Precision: among predicted phishing emails, the proportion that is truly phishing.
- Recall: among true phishing emails, the proportion correctly detected.
- F1-score: harmonic mean of precision and recall, balancing detection quality.

1.2 Core Machine Learning Concepts

- Text preprocessing: cleaning raw email content (HTML stripping, URL/email masking, normalization).
- Feature engineering: combining statistical text features (TF-IDF) with handcrafted phishing indicators.
- Model training: learning a decision boundary from labeled examples.
- Thresholding: converting phishing probability into final class decision.
- Model explainability: exposing suspicious patterns, URL counts, and confidence values.
- Hybrid inference: combining ML predictions with rule-based logic to reduce false positives.

2. Main Components of This Project

The system is organized into five primary layers:

- Data layer: datasets in data/phishing_legitimate.csv and data/spam_assassin.csv.
- Preparation layer: schema normalization, label harmonization, and text cleaning.
- Feature layer: TF-IDF vectors + phishing/legitimacy custom signals.
- Model layer: Complement Naive Bayes and Logistic Regression (baseline and v2).
- Serving layer: Flask API + web interface for interactive testing.

2.1 Data Preparation Module

The preparation stage auto-detects text and label columns across both datasets, unifies labels to binary (1 phishing, 0 legitimate), drops invalid rows, and creates cleaned_emails.csv. This makes the pipeline robust to schema differences such as label versus target.

2.2 Feature Engineering Modules

Two feature pipelines exist. Baseline features combine TF-IDF with generic suspicious terms and structural email signals. V2 features refine phishing patterns and add legitimacy indicators (professional signatures, organizational language, and context-aware cues) to reduce false alarms.

2.3 Model Training Modules

The training stage compares Complement Naive Bayes and Logistic Regression. The best model is saved as a deployable artifact. In v2, class balancing and improved features are used to optimize precision in realistic scenarios.

2.4 Inference Modules

- predictor.py: baseline inference pipeline.
- predictor_v2_improved.py: v2 model with improved feature extraction.
- predictor_ensemble.py: production hybrid model (ML score + legitimacy override logic).

2.5 Web Application Layer

The Flask application exposes /predict and serves an HTML interface where users paste

email text and inspect label, confidence, phishing probability, and reasoning signals.

3. Functional Flow of the System

3.1 Offline Training Flow

- Step 1: Load and normalize both datasets.
- Step 2: Clean text and generate cleaned_emails.csv.
- Step 3: Build feature matrices (X) and labels (y).
- Step 4: Split into train/test and train candidate models.
- Step 5: Evaluate with accuracy, precision, recall, F1, and confusion matrices.
- Step 6: Save vectorizer and best model artifacts for deployment.

3.2 Online Prediction Flow

- User submits email text via UI or API.
- Server loads artifacts (lazy loading).
- Text is cleaned and transformed into TF-IDF + custom features.
- Model outputs phishing probability.
- Threshold and/or ensemble override is applied.
- JSON response returns label, confidence, and interpretation fields.

3.3 Ensemble Decision Logic

The ensemble predictor first computes the ML phishing probability, then adjusts it with legitimacy evidence (signature, organization context, structured communication style, and non-phishing password-notification patterns). When explicit phishing patterns exist, legitimacy override is strongly reduced. This hybrid design addresses false positive errors while preserving phishing detection power.

4. Observed Performance and Practical Interpretation

From project outputs, baseline Logistic Regression reaches very strong test metrics (around 99% accuracy), and the v2 pipeline further improves precision. However, real-world false positives still appear in cases like legitimate security notifications and institutional emails. This confirms that offline metrics alone are not enough; threshold tuning and hybrid logic

are necessary in production.

5. Overview of Main Existing Solutions in the Field

Phishing defense is typically multi-layered. Existing solutions can be grouped as follows:

5.1 Rule-Based and Signature-Based Filters

- Static keyword/blocklist matching.
- Heuristic policies for suspicious headers, URLs, or attachment types.
- Strengths: fast, interpretable, low compute.
- Limitations: easy to evade, weaker against novel attacks.

5.2 Classical ML Solutions

- Algorithms: Naive Bayes, Logistic Regression, SVM, Random Forest.
- Typical features: TF-IDF, lexical and structural email indicators.
- Strengths: strong baseline performance, efficient training/deployment.
- Limitations: depends on feature quality and dataset representativeness.

5.3 Deep Learning and Transformer-Based Solutions

- Neural architectures: LSTM/CNN/BERT-family models for semantic understanding.
 - Strengths: captures context and subtle language manipulation.
 - Limitations: higher data/compute needs, harder interpretation, deployment cost.
- 5.4 Email Authentication and Infrastructure Defenses

- SPF, DKIM, and DMARC for domain and sender authenticity checks.
- Secure Email Gateways integrating reputation, sandboxing, and policy controls.
- Strengths: blocks spoofing and malicious delivery before user exposure.
- Limitations: does not fully stop social-engineering content from compromised or lookalike domains.

5.5 Hybrid Solutions (Current Best Practice)

Most robust systems combine authentication protocols, reputation intelligence, ML scoring, and expert rules. The current project follows this trend through ML plus rule-based legitimacy correction at inference time.

6. Positioning of This Project

- Category: classical ML + engineered features + hybrid ensemble override.
- Key value: practical precision improvement under false-positive pressure.
- Academic relevance: clear end-to-end lifecycle from data preparation to deployment.
- Industrial relevance: API-based serving and interpretable output fields.

7. Limitations and Future Enhancements

- Expand legitimate-email diversity to reduce dataset-induced bias.
- Add sender/domain reputation and URL intelligence features.
- Perform time-based and cross-domain validation to measure robustness drift.
- Compare against transformer baselines for semantic attack detection.
- Introduce calibration and uncertainty-aware decision policies.

Conclusion

This project illustrates how AI-powered phishing detection has become essential due to the rapid rise of sophisticated LLM-generated phishing attacks. By combining preprocessing, feature extraction, and machine learning/NLP classification, modern phishing detectors can identify malicious emails far more effectively than traditional rule-based systems. Existing solutions range from basic heuristic filters to advanced transformer-based and hybrid enterprise systems, each offering different trade-offs in interpretability, cost, and performance.